# BRAIN TUMOR CLASSIFICATION: A DEEP LEARNING APPROACH

**Talha Khan**
Student# 1009128123
Talhak.khan@mail.utoronto.ca

**Mohammadsadra Shirdarreh**
Student# 1009291286
Sadra.shirdarreh@mail.utoronto.ca

**Taha Khuwaja**
Student# 1009070345
khuwaja.taha@mail.utoronto.ca

**Saleh Mousavi**
Student# 1008848945
saleh.mousavi@mail.utoronto.ca

## ABSTRACT

The following is a project proposal for APS360: Introduction to Deep Learning.
—-Total Pages: 7

## 1 BRIEF PROJECT DESCRIPTION

Brain tumors are a prevalent disease among the elderly, this is concerning since in Canada: "the population aged 85 and older has doubled since 2001" (Hallman et al., 2022). As a result, this will likely lead to increased cases of brain tumors. The fast-paced nature of this disease makes early detection vital for a patient's survival and diagnosis is typically done by analysis of medical images (i.e. MRI scans) by experts. This task lends itself to deep learning since a model can be trained to detect brain tumors in images based on already classified images by experts. This model can be used to analyze brain scans to verify the expert's diagnosis or in circumstances where an expert is not readily accessible. This model can be a vital tool in the upcoming years as the number of cases of brain tumors is expected to increase. Thus the project proposed is a deep learning model capable of detecting tumors in brain scans, it takes as an input a brain scan in JPG format and it will classify the scan as tumourous or not tumourous.

## 2 INDIVIDUAL CONTRIBUTIONS AND RESPONSIBILITIES

The team communicates via an Apple iMessage group chat, while work sessions and meetings are conducted through FaceTime. Collaboration is facilitated through a shared Google Drive folder, where all members have access to each other's contributions. The team has designated shared notebooks for various tasks, such as data processing and training, ensuring accessibility to all team members. Task lists with assigned responsibilities are regularly distributed in the iMessage group chat.

| **Task** | **Description** | **Member** | **Deadline** |
|---|---|---|---|
| | **Data Processing** | | |
| Dataset preprocessing and standardization | Downloaded the images from the datasets, resized all images to 224x224 by padding and adjusting to the correct aspect ratio. Converted all images to JPG format. The images were then stored in a shared Google Drive folder. | Saleh | 2024/10/27 |
| Dataset splitting | Split the dataset into training, validation, and testing sets. | Talha | 2024/10/28 |
| Data augmentation | Data augment the training set to balance the number of images from each class. Applied 6 transformations to each image in the non-tumorous category. Since there were 3 times more tumorous images, only 2 random transformations from the 6 transformations were applied to each image in the tumorous category. | Taha | 2024/10/29 |
| Additional Testing Data | Gather more testing images from a different dataset. | Saleh, Taha | 2024/11/11, In Progress |
| | **Baseline Model** | | |
| Select the baseline model | Selected the SVM model as the baseline model. | Sadra | 2024/10/29 |
| Implement the baseline model | Wrote the module for the SVM model. | Sadra, Taha | 2024/11/2 |
| Train the baseline model | Trained the model with the training images from the dataset. | Sadra, Taha | 2024/11/3 |
| Record the model's performance | Recorded the model's accuracy, recall, precision, and F1 scores. | Taha | 2024/11/3 |
| Additional Evaluation | Evaluate baseline on an additional isolated dataset for testing. | Sadra | 2024/11/21, In Progress |

Table 1: Individual Contributions and Responsibilities

| Task | Description | Member | Deadline |
|---|---|---|---|
| **Primary Model** | | | |
| Select the primary model | The team decided to use transfer learning with a fully connected network as the classifier. | The team | 2024/10/28 |
| Implement the classifier model | Wrote the module for the classifier. | Saleh | 2024/10/30 |
| Make the data loaders | Wrote functions for loading the training, validation, and testing images from Google Drive into the notebook environment and making data loader objects from them. | Saleh | 2024/10/30 |
| Feature map extraction | Extracted feature maps from AlexNet and split the maps into training, validation, and testing sets. These sets were then stored back into Google Drive. | Saleh | 2024/10/30 |
| Train the primary model | Wrote a training function that trains the net with a given training data loader, logging training and validation losses and accuracies for each epoch. Loaded the extracted training, validation, and testing feature maps, created data loaders for each, and trained the model using the training function. | Saleh, Taha | 2024/10/30 |
| Record the model's performance | Modified the training function to track the model's precision, recall, and F1 score. | Talha | 2024/11/1 |
| Hyperparameter tuning | Tune hyperparameter tuning for improved model performance. | Talha | 2024/11/21, In Progress |
| Additional Evaluation | Evaluate model on an additional isolated dataset for testing. | Sadra | 2024/11/23, In Progress |

Table 2: Individual Contributions and Responsibilities

## 3 NOTABLE CONTRIBUTIONS

The project utilizes two main datasets, a 3064 image dataset from Figshare and a 3000 image dataset from Kaggle.

### 3.1 DATA PROCESSING

The figshare dataset contains brain tumor images collected between 2005 and 2010 from Nanfang Hospital, Guangzhou, China, and General Hospital, Tianjin Medical University, China (Cheng, 2017). Since this dataset only has tumorous images, an additional dataset from Kaggle, the Br35H dataset, will be used, which contains 1,500 of tumorous and non-tumorous images each (Hamada, 2021).

One of the preprocessing challenges was varying image sizes. This was resolved via a script that zero-padded images to the correct aspect ratio and then resized them to 224x224. Furthermore, each dataset had stored the images in different formats: One of the datasets provides the images in a Matlab struct variable and the other stored the images as JPG files. To ensure consistency, all of the images were converted to JPG format.

### 3.2 Data Augmentation

Due to limited resources available on the web, our dataset was relatively small, which posed challenges for effective model training and generalization. Furthermore, our dataset exhibited a significant class imbalance, with the no-tumor class being underrepresented. To address the small dataset size, we performed augmentation on the training data using the transform library from PyTorch and performed the following transformations:

1. **Rotations:** Rotates the image by a random degree between 0 and 90 degrees, adding variety in orientation and helping the model generalize across different rotations.

2. **Horizontal Flip:** Flips the image horizontally with a probability of 100%, simulating the scenarios where tumors may appear on the opposite side, thus enhancing the model's robustness.

3. **Vertical Flip:** Similar to the horizontal flip, this transformation flips the image vertically with a probability of 100%, providing further positional variability for the model.

4. **Random Affine:** Applies random affine transformations such as small translations (up to 10% of the image size). This transformation slightly shifts the image content, which aids the model in generalizing to minor positional variations of tumors.

5. **Color Jitter:** Adjusts the brightness, contrast, saturation, and hue of the images randomly, allowing the model to become more resilient to variations in lighting and color conditions.

6. **Random Blur:** Applies a random Gaussian blur to the image with a kernel size of 5 and a sigma in the range (2,5). This mimics scenarios where the image is slightly out-of-focus, ensuring that the model can still detect tumors even if the image is not sharply focused.

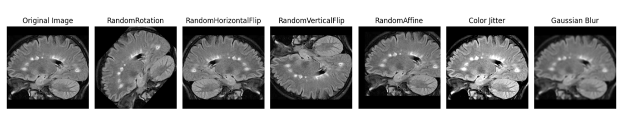


Figure 1: Example Transoformations of A Selected Image.

To address the class imbalance, we applied specific augmentations. For tumor images, we used all six transformations on every image to maximize variability within this class and enhance generalization. For non-tumor images, we applied only two randomly selected transformations per image. This approach maintained a balance between the classes, avoiding oversampling of non-tumor data while still introducing some variability. This strategy helped create a more robust model capable of generalizing across various scenarios.

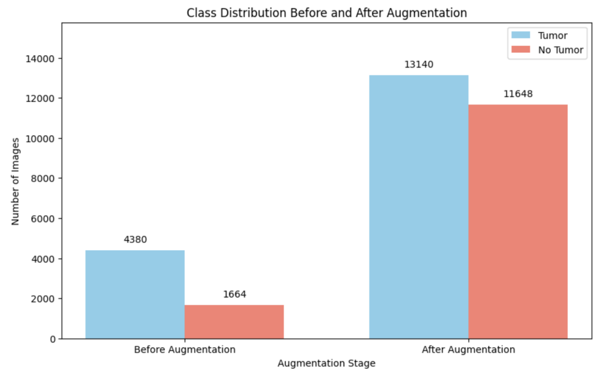


Figure 2: Class Distribution Before and After Augmentation.

### 3.3 Baseline Model

Support Vector Machine (SVM) is a machine-learning algorithm that allows the computer to classify any input that is given to it (Wahl, 2017). SVM was used as the classifier for the baseline model as it is suitable for processing complex, high-dimensional data and it is an effective classifier for detecting patterns in MRI scans with minimal preprocessing.

SVM operates by "finding the optimal hyperplane that maximizes the margin between the closest data points of opposite classes" (IBM, 2023). In n-dimensional space, multiple hyperplanes can exist that differentiate between the classes, which is why the margin between the data points is maximized to find the optimal hyperplane that represents the best decision boundary between the classes (IBM, 2023). As seen in Figure 1, the lines parallel to the optimal hyperplane are called support vectors and they cross the closest points to the decision boundary, thus they determine the position and orientation of the optimal hyperplane.

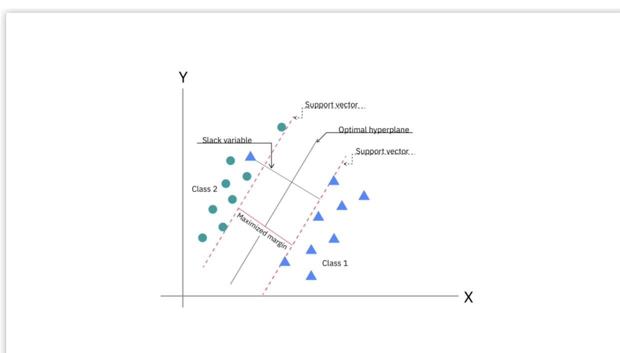


Figure 3: Optimal Hyperplane and Support Vector (IBM, 2023).

In this project, a dataset of MRI scans from Kaggle was used that contained 6044 images and the dataset was augmented to make 24,700 images. The dataset was split into three subsets: 80 for training, 10 for validation, and 10 for testing. Each MRI scan was converted into a numpy array format which makes it more efficient for handling the image data in the model. Instead of extracting individual features from the input images using convolutional layers or pretrained models like AlexNet, the baseline model fed the raw images directly into the SVM classifier without any prior feature extraction. As a result, the model relies solely on the classifier to learn patterns from the pixel values in the input MRI scans.

One of the challenges that we encountered during training the SVM on solely the MRI scans was memory constraints. To prevent the model from quickly exhausting the RAM leading to the training algorithm to crash, we used batch training by breaking the dataset into a total of 775 smaller subsets, allowing the model to process the batches sequentially. Additionally we reduced the size of the input images to 256 pixels, helping the data run efficiently without exceeding the RAM limit. Making these alterations to the model allowed for a stable and effective training process while minimizing hardware strain.

As a result of a large number of batches and a relatively large image size, training the SVM with the MRI scans presented a significant runtime challenge. Using the T4 GPU on Google Colab, training the model would take roughly three hours or more. The T4 GPUs work better for general tasks with a strong support for machine learning, however they fall short when it comes to fast processing for deep learning tasks compared to the newer GPUs. The L4 GPU, which offers optimized memory bandwidth, faster processing and data handling for deep learning tasks. This improved the training efficiency drastically reducing the training runtime by at least 50%.

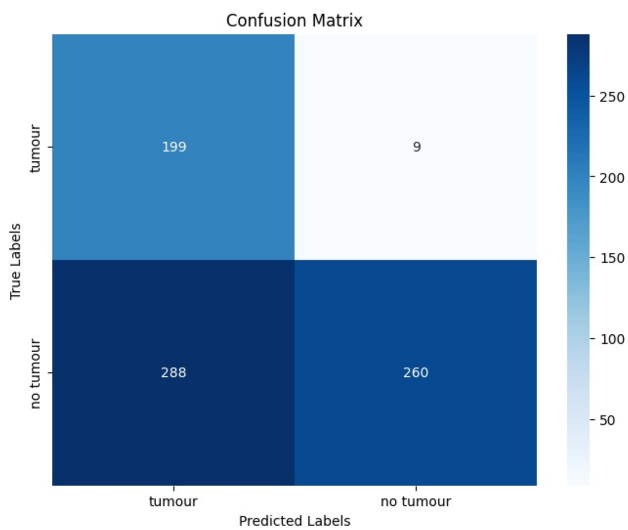


Figure 4: Illustration of the Confusion Matrix for Baseline Model.

### 3.4 PRIMARY MODEL

The primary model takes the input image (3 channels, size 224 x 244). This image is fed to the CNN of Alexnet. The Alexnet CNN consists of 5 convolutional layers. The following are the specs of the CNN:

- **Convolutional layer 1:** 96 kernels, size 11x11, stride length of 4
- **Overlapping max pooling layer:** 3x3 kernel size, stride length of 2
- **Convolutional layer 2:** 256 kernels, size 5x5, padding of 2
- **Overlapping max pooling layer:** 3x3 kernel size, stride length of 2
- **Convolutional layer 3:** 384 kernels, size 3x3, padding of 1
- **Convolutional layer 4:** 384 kernels, size 3x3, padding of 1
- **Convolutional layer 5:** 256 kernels, size 3x3, padding of 1
- **Overlapping max pooling layer:** 3x3 kernel size, stride length of 2

The main issue faced was that the feature maps produced by Alexnet took a long time to extract since the dataset is quite large, the extraction process as a whole took even longer. The output of Alexnet's feature map extractor is of size 256 x 6 x 6. This is sent to a fully connected layer with 64 neurons. The output of this layer is sent to another fully connected layer with one neuron for classification to indicate whether it was a tumorous or non-tumorous image. The ReLU (Rectified Linear Unit) is used as the activation function between the two layers. The network itself has 589,953 total trainable parameters ((256 x 6 x 6 x 64 + 64) + (64 + 1)).

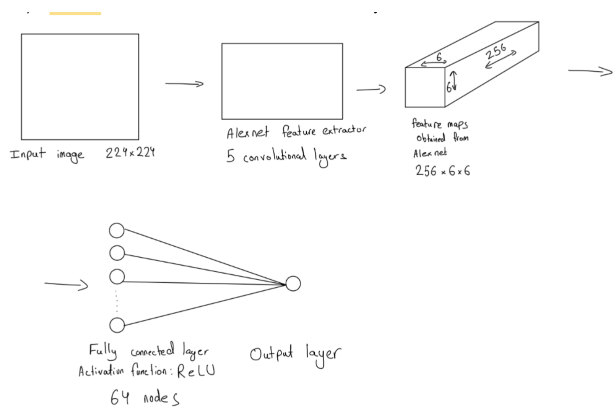


Figure 5: Illustration of the Model Architecture.

The confusion matrix (Figure 5) illustrates the effectiveness of our model in distinguishing between tumor and non-tumor. The high concentration of correct classifications in both the true positive and true negative categories shows that our model achieved high accuracy across classes with very few misclassifications.

Notably, the model shows low false negatives, meaning it successfully identifies most tumor cases which is essential in the medical field where missing a tumor can have significant consequences. Additionally, the low false positive rate reflects the model's specificity, ensuring that non-tumor images are rarely misclassified.

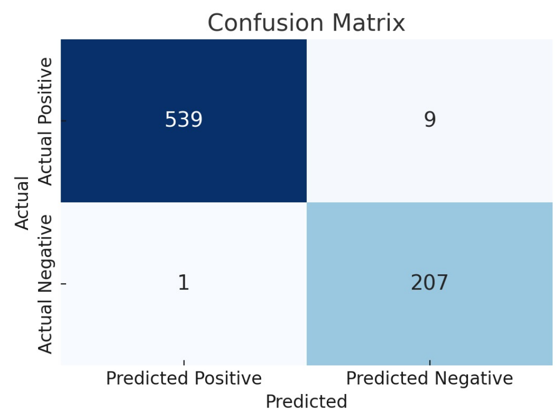


Figure 6: Illustration of the Confusion Matrix for Primary Model.

## 4 Link to Google Colab Notebook

https://colab.research.google.com/drive/1iAMDRBsypPBXkYFOSdAdEVljpirsBN9b?usp=sharing

## References

S. Hallman, S. LeVasseur, J. Bérard-Chagnon, and Canada's Centre for Demography. A portrait of canada's growing population aged 85 and older from the 2021 census. `https://www12.statcan.gc.ca/census-recensement/2021/as-sa/98-200-x/2021004/98-200-x2021004-eng.cfm`, 2022. Accessed: 2024-11-03.

IBM. What is support vector machine? `https://www.ibm.com/topics/support-vector-machine`, 2023. Accessed: 2024-11-03.

D. Wahl. I support vector machines and so should you. https://towardsdatascience.com/i-support-vector-machines-and-so-should-you-7af122b6748, 2017. Accessed: 2024-11-03.